EPFL - Computer Science

CS-451 Distributed Algorithms

Lecture 01 — 07/09/2026

Author: Jonathan Dutoit

Professor: Rachid Guerraoui

Overview

> **Definition.** A **Distributed system** is a set of processes (machines), which are Turing machines, that communicate through shared messages.

Ex.: Google. Hundreds of thousands of machines connected. One job can involve 2000 machines, and on average 10 machines go down per day.

The application needs underlying services for distributed interaction. The network is not enough. Reliability guarantees (e.g., TCP) are only offered for communication among pairs of processes, i.e., *one-to-one communication (client-server)*.

Underlying services

1. ***processes*** (abstracting computers)
2. ***channels*** (abstracting networks)
3. ***failure detectors*** (abstracting time)

> **Definition.** Processes. The distributed system is made of a finite set of ***processes***: each process models a ***sequential*** program.
>
> Processes are denoted $p_1, ..., p_N$ or p, q, r.
>
> Processes have unique ***identities*** and know each other. Every pair of processes is connected by a ***link*** through which the processes exchanges ***messages***.
>
> Every process has a ***piece*** of the distributed algorithm. Modules within the same process interact by exchanging events.
>
> ***Event based programming***:

```
upon event < Event1, att1, att2, ...> do
  // something
  trigger < Event2, att1, att2, ...>
```

Definition. ***Safety*** is a property which states that nothing bad should happen. ***Liveness*** is a property which states that something good should happen.

Example: *Tell the truth*
- Having to say something is *liveness*
- Not lying is *safety*

Fair-loss links

- ***FL1. Fair-loss***. If a message is sent infinitely often by pi to pj, and neither pi or pj crashes, them m is delivered infinitely often by pj ⇒ liveness.
- ***FL2. Finite duplication***. If a message m is sent a finite number of times by pi to pj, m is delivered a finite number of times by pj. ⇒ liveness.
- ***FL3. No creation***. No message is delivered unless it was sent. ⇒ safety.

Stubborn links

- ***SL1. Stubborn delivery***. If a process pi sends a message m to a correct process pj, and pi does not crash, then pj delivers m an infinite number of times.
- ***SL2. No creation***. No message is delivered unless it was sent.

Implements: SturbbornLinks (sp2p).
Uses: FairLossLinks (flp2p).

```
upon event <sp2pSend, dest, m> do
  while (true) do
    trigger <flp2pSend, dest, m>;
  upon event <flp2pDeliver, src, m> do
    trigger (sp2pDeliver, src, m>;
```

Note: by the underlying link (fair-loss links), we know that if we send a message an infinite number of times, then the message is delivered infinitely. For delivering, the message comes first from the underlying link (fair-loss), and then it's transferred to the upper link (stubborn).

Reliable (Perfect) links

- ***PL1. Validity***. If pi and pj are correct, then every message sent by pi to pj is eventually delivered by pj.
- ***PL2. No duplication***. No message is delivered (to a process) more than once.
- ***PL3. No creation***. No message is delivered unless it was sent.

Implements: PerfectLinks (pp2p)
Uses: StubbornLinks (sp2p).

```
upon event < Init> do delivered := ∅
upon event < pp2pSend, dest, m>;
  trigger < sp2pSend, dest, m>;
upon event < sp2pDeliver, src, m> do
  if m ∉ delivered then
    trigger < pp2pDeliver, src, m>;
    add m to delivered;
```

Roughly speaking, reliable links ensure that messages exchanged between correct processes are ***not lost***.